

AUTHORED BY: ARPIT RAJ, LNMIIT JAIPUR

Date: July 23, 2026

Chapter: Path with Maximum Probability (LeetCode 1514)

Below is the optimized graph traversal implementation to find the path with the maximum probability of success. The algorithm heavily relies on a modified version of Dijkstra's algorithm, utilizing a max-heap and tracking maximum probabilities instead of minimum costs.

```

class Solution {
public:
    double maxProbability(int n, vector<vector<int>>& edges,
vector<double>& succProb, int start_node, int end_node) {

        //step1 build adjacency

        vector<vector<pair<int,double>>> adj(n);
        for(int i=0;i<edges.size();i++){
            int u=edges[i][0];
            int v=edges[i][1];
            double p=succProb[i];

            adj[u].push_back({v,p});
            adj[v].push_back({u,p});
        }

        //step 2 build max heap to store {probab,node}
        //keep a maxprobab vector to keep track of highest
        probab to reach a node

        priority_queue<pair<double,int>> pq;
        vector<double> maxp(n,0);

        maxp[start_node]=1;
        pq.push({1,start_node});

        while(!pq.empty()){
            int currnode=pq.top().second;
            double currprob=pq.top().first;
            pq.pop();

            if(currprob<maxp[currnode]) continue;
            for(auto &it:adj[currnode]){
                int newnode=it.first;
                double cost=it.second;
                double newprob=currprob*cost;

                if(newprob>maxp[newnode]){
                    maxp[newnode]=newprob;
                    pq.push({newprob,newnode});
                }
            }
        }
    }
}

```

```
    }  
  
    return maxp[end_node];  
}  
};
```